1. Problem statement and our understanding of it

For generations, Indians, blind or sighted, have shopped by talking: "Omnikart, a good three-litre cooker under fifteen hundred." The shopkeeper asks one question, shows two or three options, says which one sells best, and tells you the truth about the handle.

Online shopping replaced that conversation with a visual catalogue. That single change locked out people who cannot see. A screen reader does not fix it: it makes a blind shopper listen to the whole shop, one element at a time. Forty filter checkboxes, a carousel, "Sponsored", a 900-word spec sheet, then 2,000 reviews. The information is technically there, but nobody can hold it in their head.

So we read the problem as a conversation problem, not a screen problem. Our user needs four things the catalogue never gives them: (a) fewer options, chosen for them, (b) the one or two facts that matter, in plain words, (c) honest opinions from people like them, and (d) certainty before money leaves their account. They need this in Hindi or Telugu as easily as in English, and the same gap hits two more groups hard: people with limited hand movement (small targets, long forms) and older shoppers (dense pages, fast speech, too many steps).

2. Idea / abstract

Omnikart is a voice shopping assistant sitting on top of a demo e-commerce website we build ourselves. You speak in English, Hindi or Telugu; it talks back the way a good shopkeeper does. It asks one clarifying question, offers three options at a time, sums up hundreds of reviews into what people liked, what they did not, and a verdict, describes product photos and size charts in words, compares two products in a sentence, and reads back every number before you pay. The website itself is fully accessible, so a sighted helper sees exactly what Omnikart is talking about.

Visual impairment is our primary condition. We add hand-motor impairment and ageing. The app does not ask "what disability do you have". It notices signals and offers a change once, for example: "Would you like me to speak slower and show one thing at a time?" The user can say no, and it never labels them.

Ideas we think are new:

- **Community reviews by and for disabled shoppers.** Shoppers with disabilities can leave a short review answering things normal reviews skip: "Can I tell the buttons apart by touch?", "Is the app usable with TalkBack?", "Did the size chart match?" These show up as their own section on the product page and Omnikart reads them first.
- **"Explain simpler" that translates, not shortens.** Every output has a simpler version that converts specs into everyday meaning. "5,000 mAh" becomes "about two days per charge". "1.5 tonne AC" becomes "right for a medium bedroom". Same facts, no jargon.
- **One large fact per screen.** For ageing and low-vision users, the screen shows one large fact at a time (price, then delivery date, then rating) instead of a crowded page.
- **Polite turn-taking with a wind chime.** The user can interrupt Omnikart any time; Omnikart never talks over the user. When the user stops speaking for two seconds, a soft two-note wind chime says "I heard you, thinking now", so a blind user is never left wondering if the app is listening.
- **Decision log box.** A visible panel logs every step and decision ("ASR heard X, confidence 0.62, asked to confirm"; "price read from catalogue, not from the LLM"). Judges and helpers can see why Omnikart did what it did.

3. High-level approach and architecture

Pipeline for every turn, all speech and translation through Bhashini:

```
Mic -> Bhashini ASR (en / hi / te) -> Bhashini Translation to English -> LLM agent
                                (typed tool calls on catalogue)
-> Accessible output layer -> Bhashini Translation to user language -> Bhashini TTS
                                    + captions on screen
```

Photos and size-chart images go through Bhashini OCR plus a vision model before the same path. If Bhashini TTS fails, we fall back to the browser voice and captions are always on.

- **Demo e-commerce website.** A small mock store (about 15 products, 4 categories) with real headings, labelled buttons, tables, a skip link and a live region. Three "hero" products have 50+ mixed-language reviews, a photo and a size chart. It runs as one Next.js web app on a laptop and on Android Chrome.

- **Agent with guard rails.** The LLM only picks from typed actions: search, filter, open product, compare, add to cart, read cart, checkout, track, return. Prices, ratings, dates and stock are always filled in from catalogue data by templates. The LLM never writes a number. Review and description text is treated as untrusted data.
- **Accessible output layer.** Specs to "top 5 things to know"; reviews to pros, cons, verdict and "based on N reviews"; photos to colour, shape, material and visible text, with "looks like" when unsure; size charts to "for size M, chest 38 to 40 inches"; two-product comparison ending with "the main difference is...". Each has a raw view and an accessible view for side-by-side demo.
- **Safe checkout.** Checkout reads back items, quantity, address, delivery date, total and payment method. It needs the spoken phrase "Yes, place order" and a labelled button. Silence, "maybe", or a low-confidence hearing does not count. Demo payment only.
- **Condition inference and adaptation.** Visual: voice-first, full image descriptions. Hand-motor (signals: mis-taps, long presses): single-key scanning, big targets, merged steps, push-to-talk. Ageing (signals: repeated "slower", "what?", long pauses): slower speech, large text, one fact per screen, "Step 1 of 3", longer wait before the chime. Stricter setting wins when conditions combine.
- **Graceful failure.** Unclear speech: "I heard ten. Is that right?". Numbers, sizes and quantities are always read back. Network drop: "Connection lost, I will retry when you are back online", and the request is queued. Numbers like ₹1,299 and XL are protected so translation cannot change them. Everything is spoken and captioned in the user's language.

We do not give medical, legal or financial advice; Omnikart reads what the seller says and stops there. Every external API, model and dataset is listed in DECLARATIONS.md.

4. Planned testing methodology

- **Automated.** Unit tests for error handling, number protection in translation, and the turn-taking rules (two-second pause gives one chime; a cough gives none; interrupting stops speech within 150 ms; interrupting a price `readback cancels the pending "yes"). A fault switch simulates network loss, timeouts and rate limits so we can check each spoken recovery.
- **Accessibility checks.** axe-core in CI on every page, plus manual runs with TalkBack on Android and keyboard only. No information by colour or image alone.
- **Language and accuracy.** A scripted 10-turn shopping conversation (find, filter, compare, add, checkout, track) run end to end in English, Hindi and Telugu, including cross-language (speak Hindi, hear Telugu) and code-mixed Hinglish. We check that numbers survive translation and that summaries only say what the source says.
- **Condition walkthroughs.** One task per condition: a blindfolded tester buys a cooker by voice; a tester using only one key (or one finger in a mitten) adds to cart and checks out; an older family member orders using slow speech and large text. We note time taken, number of errors and repeats, and ask: "Did you trust the final price before paying?"
- **Real users.** If time allows, we will test with at least one blind user and one older user, and share their feedback and quotes in the final submission.